ASSIGNMENT – 7.5

2303A510H5

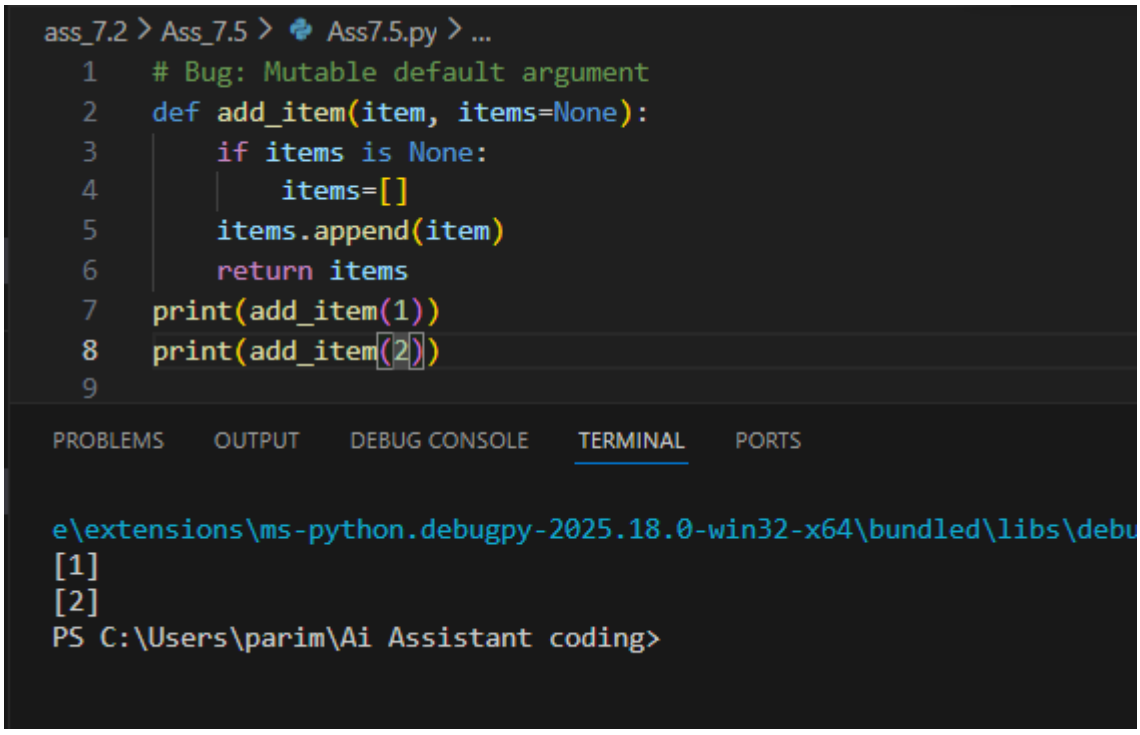
BATCH – 30

Task 1 (Mutable Default Argument – Function Bug)

Task: Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it.

```
# Bug: Mutable default argument

def add_item(item, items=[]):

    items.append(item)

return items

print(add_item(1))

print(add_item(2))
```

Expected Output: Corrected function avoids shared list bug.



Task 2 (Floating-Point Precision Error)

Task: Analyze given code where floating-point comparison fails. Use AI to correct with tolerance.

Bug: Floating point precision issue def

check_sum():

return (0.1 + 0.2) == 0.3

print(check_sum())

Expected Output: Corrected function

```
# Bug: Floating point precision issue def
def check_sum():
    return (0.1+0.2) == 0.3
print(check_sum())
```

```
e\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '5043
False
PS C:\Users\parim\Ai Assistant coding>
```

Task 3 (Recursion Error – Missing Base Case)

Task: Analyze given code where recursion runs infinitely due to missing base case. Use AI to fix.

Bug: No base case def

countdown(n):

print(n)

return countdown(n-1) countdown(5)

Expected Output : Correct recursion with stopping condition.

```
#Analyze given code where recursion runs infinitely due to missing base case.
def countdown(n):
    print(n)
    if n ==0:
        return
    countdown(n-1)
countdown(5)
```

```
5
4
3
2
1
0
PS C:\Users\parim\Ai Assistant coding>
```

Task 4 (Dictionary Key Error)

Task: Analyze given code where a missing dictionary key causes error. Use AI to fix it.

Bug: Accessing non-existing key def

get_value():

data = {"a": 1, "b": 2}

return data["c"]

print(get_value())

Expected Output: Corrected with .get() or error handling.

```python
#Bug:Accessing non-existing key def
def get_value():
    dats= {"a":1,"b":2}
    return dats.get("c", None)
print(get_value())
```

```
PS C:\Users\parim\Ai Assistant coding> c:; cd 'c:\Users\parim
e\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\lib
None
PS C:\Users\parim\Ai Assistant coding>
```

Task 5 (Infinite Loop – Wrong Condition)

Task: Analyze given code where loop never ends. Use AI to detect and fix it.

Bug: Infinite loop def

loop_example():

i = 0

while i < 5:

print(i)

Expected Output: Corrected loop increments i.

```python
#Analyze given code where loop never ends. use AI to detect and fix it.
def loop_example():
    i = 0
    while i<5:
        print(i)
        i += 1
loop_example()
```

```
                                                ^C
PS C:\Users\parim\Ai Assistant coding>
PS C:\Users\parim\Ai Assistant coding>  c:; cd 'c:\Users\
e\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle
0
1
2
3
4
PS C:\Users\parim\Ai Assistant coding>
```

Task 6 (Unpacking Error – Wrong Variables)

Task: Analyze given code where tuple unpacking fails. Use AI to fix it.

Bug: Wrong unpacking

a, b = (1, 2, 3)

Expected Output: Correct unpacking or using _ for extra values.

```python
#task_6
#Analyze given code where tuple unpacking fails .use Ai ti fix it.
a,b,c=(1,2,3)
print(a,b)
```

```
PS C:\Users\parim\Ai Assistant coding>  c:; cd  c:\Users\parim\Ai Assista
e\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\l
1 2
PS C:\Users\parim\Ai Assistant coding>
```

Task 7 (Mixed Indentation – Tabs vs Spaces)

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it.

Bug: Mixed indentation

def func():

x = 5

y = 10

return x+y

Expected Output : Consistent indentation applied

```
#task_7
def func():
    x = 5
    y = 10
    return x + y
print(func())
```

```
PS C:\Users\parim\Ai Assistant coding>  c:; cd 'c:\Users\par
e\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\l
15
PS C:\Users\parim\Ai Assistant coding>
```

Task 8 (Import Error – Wrong Module Usage)

Task: Analyze given code with incorrect import. Use AI to fix.

Bug: Wrong import import

maths print(maths.sqrt(16))

Expected Output: Corrected to import math

```
#task_8
#Analyze gievn code with incorrect import
import math
print(math.sqrt(16))
```

```
 Assistant coding\ass_7.2\Ass_7.5\Ass7.5.py'
4.0
PS C:\Users\parim\Ai Assistant coding> ^C
```

Task 9 (Unreachable Code – Return Inside Loop)

Task: Analyze given code where a return inside a loop prevents full iteration. Use AI to fix it.

Bug: Early return inside loop

def total(numbers): for n in

numbers: return n

print(total([1,2,3]))

Expected Output: Corrected code accumulates sum and returns after loop.

```
#Analyze given code where a return inside a loop prevents full iteration
def total(numbers):
    sum_value=0
    for n in numbers:
        sum_value+=n
    return sum_value
print(total([1,2,3]))
```

```
PS C:\Users\parim\Ai Assistant coding>  c:; cd  c:\Users\parim\
e\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs
6
PS C:\Users\parim\Ai Assistant coding>
```

Task 10 (Name Error – Undefined Variable)

Task: Analyze given code where a variable is used before being defined. Let AI detect and fix the error.

Bug: Using undefined

variable def calculate_area():

return length * width

print(calculate_area())

Requirements:

- Run the code to observe the error.
- Ask AI to identify the missing variable definition.
- Fix the bug by defining length and width as parameters.
- Add 3 assert test cases for correctness.

Expected Output :

- Corrected code with parameters.
- AI explanation of the bug.

Successful execution of assertions.

```
#task_10
#Analyze given code where a variable is used before being defined
def calculate_area(length,width):
    return length * width
print(calculate_area(5, 10))
```

```
PS C:\Users\parim\Ai Assistant coding>  c:; cd 'c:\Users\parim\Ai As
e\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debu
50
PS C:\Users\parim\Ai Assistant coding>
```

Task 11 (Type Error – Mixing Data Types Incorrectly)

Task: Analyze given code where integers and strings are added incorrectly. Let AI detect and fix the error.

Bug: Adding integer and string

def add_values(): return 5 +

"10" print(add_values())

Requirements:

- Run the code to observe the error.
- AI should explain why int + str is invalid.
- Fix the code by type conversion (e.g., int("10") or str(5)).
- Verify with 3 assert cases.

Expected Output #6:

- Corrected code with type handling.
- AI explanation of the fix.

Successful test validation.

```
#task_11
#Analyze given code where integers and strings are added incorrectly.
def add_value():
    return 5  + int("10")
print(add_value())
```

```
PS C:\Users\parim\Ai Assistant coding>  c:; cd 'c:\Users\parim\Ai Assistant
e\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\laun
15
PS C:\Users\parim\Ai Assistant coding>
```

Task 12 (Type Error – String + List Concatenation)

Task: Analyze code where a string is incorrectly added to a list.

Bug: Adding string and list

def combine(): return

"Numbers: " + [1, 2, 3]

print(combine())

Requirements:

- Run the code to observe the error.
- Explain why str + list is invalid.

- Fix using conversion (str([1,2,3]) or " ".join()).
- Verify with 3 assert cases.

Expected Output:

- Corrected code
- Explanation

Successful test validation

```
#task_12
#Analyze code where a string is incorrectly added to a list
def combine():
    return "Numbers:" +str([1, 2, 3])
print(combine())
```

```
e\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debu
Numbers:[1, 2, 3]
PS C:\Users\parim\Ai Assistant coding>
```

Task 13 (Type Error – Multiplying String by Float)

Task: Detect and fix code where a string is multiplied by a float. # Bug: Multiplying string by float def repeat_text(): return "Hello" * 2.5 print(repeat_text())

Requirements:

- Observe the error.
- Explain why float multiplication is invalid for strings.
- Fix by converting float to int.

Add 3 assert test cases

```
#task13
#Detect and fix code where a string is multiplied by a float
def repeat_text():
    return "Hello" * int(2.5)
print(repeat_text())
```

```
e\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\
HelloHello
PS C:\Users\parim\Ai Assistant coding>
```

Task 14 (Type Error – Adding None to Integer)

Task: Analyze code where None is added to an integer.

Bug: Adding None and integer

def compute(): value = None

return value + 10

print(compute())

Requirements:

- Run and identify the error.
- Explain why NoneType cannot be added.
- Fix by assigning a default value.
- Validate using asserts.

```
#task_14
#Analyze code where none is added to an ineteger
def compute():
    value=0
    return value + 10
print(compute())
```

```
e\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\l
10
PS C:\Users\parim\Ai Assistant coding>
```

Task 15 (Type Error – Input Treated as String Instead of Number)

Task: Fix code where user input is not converted properly.

Bug: Input remains string

def sum_two_numbers():

  a = input("Enter first number: ")

b = input("Enter second number: ")

  return a + b

print(sum_two_numbers())

Requirements:

- Explain why input is always string.
- Fix using int() conversion.
- Verify with assert test cases.

```python
#task_15
#fix code where user input is not converted properly
def sum_two_numbers():
    a = int(input("Enter first number:"))
    b = int(input("Enter second number:"))
    return a + b
print(sum_two_numbers())
```

```
e\extensions\ms-python.debugpy-2025.18.0-win32-x64\bu
Enter first number:3
Enter second number:5
8
```